

Let's Get Rusty! -- Rust 入门指北: 语言简介

1. 简介

Rust 是由 Mozilla（现由 Rust 基金会维护和推广）发起并持续发展的系统级编程语言，旨在提供**高性能并发、内存安全和现代语言特性**。

它具有类似 C/C++ 语言的执行效率，但同时提供了一套独特、严格的**编译时安全模型**，能够在编译阶段检测出许多潜在的内存错误和数据竞争问题。

下面是对 Rust 的主要特点、设计理念和生态做一个较全面的介绍。

1. 所有权模型（Ownership）

- Rust 最具代表性的概念。通过“所有权”和“借用”（borrowing）的方式，在编译时实现对内存访问安全的静态检查。
- 每个值在任意时刻只有唯一的所有者（owner），所有者离开其作用域后，值会被自动释放。
- 当需要使用同一个值时，可以通过“借用”来获取对该值的引用。

借用有两种类型：

- 可变引用（&mut T）
- 不可变引用（&T）
- 借用规则：
 - i. 任意时刻，只能有一个可变引用（&mut T），或者同时拥有若干个不可变引用（&T），而不能两者兼得。
 - ii. 借用的生命周期不能超过所有者的生命周期。
- 通过所有权模型，Rust 在编译时就避免了内存泄漏、悬垂指针（dangling pointer）和数据竞争等问题。

2. 不可变与可变语义

- Rust 默认所有变量都是不可变的（immutable），需要显式使用 `mut` 关键字来声明可变量。
- 这种“默认不可变”的设计鼓励开发者尽量以函数式思维编写整体不可变的逻辑，从而减少不必要的副作用。

3. 生命周期（Lifetimes）与借用检查器（Borrow Checker）

- 由于所有权和借用需要在编译时受到严格限制，Rust 在函数签名和引用场景中使用“生命周期标注”（lifetime annotations）来帮助编译器确定对象引用的合法时间范围。

- 借用检查器会在编译阶段检测不合法的引用或借用，确保代码在运行前就能通过编译时的内存安全检查。

4. 并发与多线程

- Rust 从语言层面支持线程安全，提供了多种并发编程方式：
 - 使用标准库中的通信原语，如 `std::sync::Mutex`、`std::sync::Arc`。
 - 更高级抽象如并行迭代器（如 crates.io 上的 Rayon 库）等。
- Rust 编译器可以在编译期检测到数据竞争和不安全的多线程访问，鼓励以“安全并发”的方式编写程序。

5. 模块化与包管理

- Rust 通过“模块（module）”和“包（crate）”实现多文件、多库的组织方式。
- Rust 官方包管理器 `cargo` 提供了从依赖管理、构建、测试到打包的一系列操作，得益于 crates.io 生态，一个 `Cargo.toml` 就能将依赖库轻松整合进项目。

6. 函数式与泛型编程

- Rust 函数式编程风格：迭代器、闭包（closure）、模式匹配（match）、枚举（enum）等。
- Rust 提供强大的泛型系统，并支持特征（trait）用于实现面向对象类似概念（如接口、多态）。
- 零成本抽象：泛型及特征在编译后会进行“单态化”（monomorphization），在大多数场景下不损失性能。

7. 安全与不安全代码（Unsafe）

- Rust 默认所有代码都是“安全的”（safe），但在需要直接操作内存地址、执行底层硬件操作等场景时可以使用 `unsafe` 关键字。
- `unsafe` 并不代表一定不安全，而是暗示编译器不再通过借用检查等特性来保证内存安全，开发者需要自己保证这部分代码的正确性。
- 通过严格限制 `unsafe` 代码块的范围，Rust 大幅度降低了常见内存相关 bug 的风险。

2. 语法

2.1 变量

使用 `let` 关键词，遵循 `snake_case`

```
1 fn main() {  
2     // 自动推断出类型  
3     let x = 13;  
4     println!("{}", x);  
}
```

```

5
6    // 显式声明类型
7    let x: f64 = 3.14159;
8    println!("{}", x);
9
10   // 也可以先声明后初始化
11   let x;
12   x = 0;
13   println!("{}", x);
14 }

```

2.1.1 变量可修改性

变量的值分为两种类型：

- **可变的** 编译器允许对变量进行读取和写入。
- **不可变的** 编译器只允许对变量进行读取。

可变值用 `mut` 关键字表示

```

1  fn main() {
2      let mut x = 42;
3      println!("{}", x);
4      x = 13;
5      println!("{}", x);
6  }

```

2.1.2 常量

`const` 关键词，常量名总是遵循 **全大写蛇形命名法** `SCREAMING_SNAKE_CASE`

```

1  const PI: f32 = 3.14159;
2
3  fn main() {
4      println!(
5          "To make an apple {} from scratch, you must first create a universe.",
6          PI
7      );
8  }

```

2.1.1 基本类型

- 布尔型 - `bool` 表示 true 或 false

- 无符号整型 - `u8` `u32` `u64` `u128` 表示非负整数
- 有符号整型 - `i8` `i32` `i64` `i128` 表示整数
- 指针大小的整数 - `usize` `isize` 表示内存中内容的索引和大小
- 浮点数 - `f32` `f64`
- 元组 (tuple) - `(value, value, ...)`
- 数组 - 在编译时具有固定长度的相同元素的集合
- 切片 (slice) - 在运行时已知长度的相同元素的集合
- `str` (string slice) - 在运行时已知长度的文本

也可以通过将类型附加到数字的末尾来明确指定数字类型 (如 `13u32` 和 `2u8`)

```
1 fn main() {
2     let x = 12; // 默认情况下, 这是i32
3     let a = 12u8;
4     let b = 4.3; // 默认情况下, 这是f64
5     let c = 4.3f32;
6     let bv = true;
7     let t = (13, false);
8     let sentence = "hello world!";
9     println!(
10         "{} {} {} {} {} {} {} {}",
11         x, a, b, c, bv, t.0, t.1, sentence
12     );
13 }
```

基本类型转换

使用 `as` 关键字

```
1 fn main() {
2     let a = 13u8;
3     let b = 7u32;
4     let c = a as u32 + b;
5     println!("{}", c);
6
7     let t = true;
8     println!("{}", t as u8);
9 }
```

2.1.2 数组

数组是所有相同类型数据元素的固定长度集合。

一个数组的数据类型是 `[T;N]`，其中 `T` 是元素的类型，`N` 是编译时已知的固定长度。

可以使用 `[x]` 运算符提取单个元素，其中 `x` 是所需元素的 `usize` 索引（从 0 开始）。

```
1 fn main() {
2     let nums: [i32; 3] = [1, 2, 3];
3     println!("{:?}", nums);
4     println!("{}", nums[1]);
5 }
```

2.1.3 函数

函数可以有 0 个或者多个参数。

在这个例子中，`add` 接受类型为 `i32`（32 位长度的整数）的两个参数。

函数名总是遵循 蛇形命名法（`snake_case`）。

```
1 fn add(x: i32, y: i32) -> i32 {
2     return x + y;
3 }
4
5 fn main() {
6     println!("{}", add(42, 13));
7 }
```

多个返回值

函数可以通过元组来返回多个值，元组元素可以通过索引来获取。

```
1 fn swap(x: i32, y: i32) -> (i32, i32) {
2     return (y, x);
3 }
4
5 fn main() {
6     // 返回一个元组
7     let result = swap(123, 321);
8     println!("{}", result.0, result.1);
9
10    // 将元组解构为两个变量
11    let (a, b) = swap(result.0, result.1);
12    println!("{}", a, b);
13 }
```

返回空值

如果没有为函数指定返回类型，它将返回一个空的元组，即 *unit* / 单元，空的元组用 `()` 表示。直接使用 `()` 的情况相当不常见，但它经常会出现（比如作为函数返回值）。

```
1 fn make_nothing() -> () {
2     return ();
3 }
4
5 // 返回类型隐含为 ()
6 fn make_nothing2() {
7     // 如果没有指定返回值，这个函数将会返回 ()
8 }
9
10 fn main() {
11     let a = make_nothing();
12     let b = make_nothing2();
13
14     // 打印a和b的debug字符串，因为很难去打印空
15     println!("The value of a: {:?}", a);
16     println!("The value of b: {:?}", b);
17 }
```

2.2 基本控制流

2.2.1 条件控制: if & else

Rust 的条件判断没有括号，所有常见的逻辑运算符仍然适用：

`==`，`!=`，`<`，`>`，`<=`，`>=`，`!`，`||`，`&&`

⚠ 因为if/else 在rust 也是表达式，意味着有值返回

```
1 fn main() {
2     let x = 42;
3     if x < 42 {
4         println!("less than 42");
5     } else if x == 42 {
6         println!("is 42");
7     } else {
8         println!("greater than 42");
9     }
10 }
```

2.2.2 循环控制

2.2.2.1 loop

`loop` 无限循环，可以使用 `break` 跳出；

`loop` 也是表达式

```
1  fn main() {
2      let mut x = 0;
3      loop {
4          x += 1;
5          if x == 42 {
6              break;
7          }
8      }
9      println!("{}", x);
10
11     let mut y = 0;
12     let v = loop {
13         x += 1;
14         if y == 13 {
15             break "found the 13";
16         }
17     };
18     println!("from loop: {}", v);
19 }
20
```

2.2.2.2 while

使用 `while` 进行带条件的循环控制

`while` 也是表达式

```
1  fn main() {
2      let mut x = 0;
3      while x != 42 {
4          x += 1;
5      }
6  }
```

2.2.2.3 for

Rust 的 `for` 循环是一个强大的升级。它遍历来自计算结果为迭代器的任意表达式的值。

```
1 fn main() {
2     for x in 0..5 {
3         println!("{}", x);
4     }
5
6     for x in 0..=5 {
7         println!("{}", x);
8     }
9 }
10
```

- `..` 运算符创建一个可以生成包含起始数字、但不包含末尾数字的数字序列的迭代器。
- `..=` 运算符创建一个可以生成包含起始数字、且包含末尾数字的数字序列的迭代器。

2.2.2.4 match

c/cpp 中通过 `switch` 进行匹配，rust 没有这个，但是有 `match`

`match` 是穷举的，意为所有可能的值都必须被考虑到

Rust 中最常见的模式就是 match pattern!

```
1 fn main() {
2     let x = 42;
3
4     match x {
5         0 => {
6             println!("found zero");
7         }
8         // 我们可以匹配多个值
9         1 | 2 => {
10            println!("found 1 or 2!");
11        }
12        // 我们可以匹配迭代器
13        3..=9 => {
14            println!("found a number 3 to 9 inclusively");
15        }
16        // 我们可以将匹配数值绑定到变量
17        matched_num @ 10..=100 => {
18            println!("found {} number between 10 to 100!", matched_num);
19        }
20        // 这是默认匹配，如果没有处理所有情况，则必须存在该匹配
21        _ => {
```



```
22         println!("found something else!");
23     }
24 }
25 }
```

2.2.3 概念: 语句和表达式

Rust是一个基于表达式的语言，不过它也有语句。Rust只有两种语句：

- 声明语句
- 表达式语句

其他的都是表达式。

基于表达式是函数式语言的一个重要特征，表达式总是返回值。

2.2.3.1 声明语句

声明语句可以分为两种，一种为变量声明语句，另一种为item声明语句

变量声明语句

主要是指 `let` 语句，如：

```
1  let a = 8;
2  let b: Vec::f64 = Vec::new();
```

由于let是语句，所以不能将let语句赋给其他值。如下形式是错误的：

```
1  let b = (let a = 8);
```

item 语句

是指函数（function）、结构体（structure）、类型别名（type）、静态变量（static）、特质（trait）、实现（implementation）或模块（module）的声明。这些声明可以嵌套在任意块（block）中。

这里可以看官方文档了解， 不作展开

[The Rust Reference](#)

2.2.3.2 表达式语句

由一个表达式和一个分号组成，即在表达式后面加一个分号就将一个表达式转变为了一个语句。所以，有多少种表达式，就有多少种表达式语句

这里可以看官方文档了解， 不作展开

Expressions

2.2.4 总结

`if`，`match`，函数，以及作用域块都有一种返回值的独特方式。

如果 `if`、`match`、函数或作用域块中的最后一条语句是不带 `;` 的表达式，Rust 将把它作为一个值从块中返回。

这是一种创建简洁逻辑的好方法，它返回一个可以放入新变量的值，还允许 `if` 语句像简洁的三元表达式一样操作。

```
1 fn example() -> i32 {
2     let x = 42;
3     // Rust的三元表达式
4     let v = if x < 42 { -1 } else { 1 };
5     println!("from if: {}", v);
6
7     let food = "hamburger";
8     let result = match food {
9         "hotdog" => "is hotdog",
10        // 当它只是一个返回表达式时，大括号是可选的
11        _ => "is not hotdog",
12    };
13    println!("identifying food: {}", result);
14
15    let v = {
16        // 这个作用域块可以得到一个不影响函数作用域的结果
17        let a = 1;
18        let b = 2;
19        a + b
20    };
21    println!("from block: {}", v);
22
23    // 在最后从函数中返回值的惯用方法
24    v + 4
25 }
26
27 fn main() {
28     println!("from function: {}", example());
29 }
30
```

2.3 基本数据结构

2.3.1 结构体

和c/cpp一样

```
1 struct SeaCreature {
2     // String 是个结构体
3     animal_type: String,
4     name: String,
5     arms: i32,
6     legs: i32,
7     weapon: String,
8 }
```

2.3.1.1 类元祖结构体

```
1 struct Location(i32, i32);
2
3 fn main() {
4     // 这仍然是一个在栈上的结构体
5     let loc = Location(42, 32);
6     println!("{}", loc.0, loc.1);
7 }
8
```

2.3.1.2 类单元结构体

结构体也可以没有任何字段。一个 *unit* 是空元组 `()` 的别称。这就是为什么，此类结构体被称为 **类单元**。

```
1 struct Marker;
2
3 fn main() {
4     let _m = Marker;
5 }
6
```

2.3.2 内存

Rust 程序有 3 个存放数据的内存区域：

- **数据内存** 对于固定大小和**静态**（即在整个程序生命周期中都存在）的数据。

考虑一下程序中的文本（例如“Hello World”），该文本的字节只能读取，因此它们位于该区域中。编译器对这类数据做了很多优化，由于位置已知且固定，因此通常认为编译器使用起来非常快。

- **栈内存** 对于在函数中声明为变量的数据。

在函数调用期间，内存的位置不会改变，因为编译器可以优化代码，所以栈数据使用起来比较快。

- **堆内存** 对于在程序运行时创建的数据。

区域中的数据可以添加、移动、删除、调整大小等。由于它的动态特性，通常认为它使用起来比较慢，但是它允许更多创造性的内存使用。当数据添加到该区域时，我们称其为**分配**。从本区域中删除数据后，将其称为**释放**。

```
1  struct SeaCreature {
2      animal_type: String,
3      name: String,
4      arms: i32,
5      legs: i32,
6      weapon: String,
7  }
8
9  fn main() {
10     // SeaCreature的数据在栈上
11     let ferris = SeaCreature {
12         // String 结构体也在栈上，
13         // 但也存放了一个数据在堆上的引用
14         animal_type: String::from("螃蟹"),
15         name: String::from("Ferris"),
16         arms: 2,
17         legs: 4,
18         weapon: String::from("大钳子"),
19     };
20
21     let sarah = SeaCreature {
22         animal_type: String::from("章鱼"),
23         name: String::from("Sarah"),
24         arms: 8,
25         legs: 0,
26         weapon: String::from("无"),
27     };
28     //Ferris 和 Sarah 有在程序中总是固定的位置的数据结构，所以它们被放在栈上
29     println!(
30         "{} 是只{}。它有 {} 只胳膊 {} 条腿，还有一个{}。",
31         ferris.name, ferris.animal_type, ferris.arms, ferris.legs,
32         ferris.weapon
```

```

32     );
33     println!(
34         "{} 是只{}". 它有 {} 只胳膊 {} 条腿。它没有杀伤性武器...",
35         sarah.name, sarah.animal_type, sarah.arms, sarah.legs
36     );
37 }
38

```

2.3.3 枚举

enum 关键字

```

1  #![allow(dead_code)] // this line prevents compiler warnings
2
3  enum Species {
4      Crab,
5      Octopus,
6      Fish,
7      Clam
8  }
9
10 struct SeaCreature {
11     species: Species,
12     name: String,
13     arms: i32,
14     legs: i32,
15     weapon: String,
16 }
17
18 fn main() {
19     let ferris = SeaCreature {
20         species: Species::Crab,
21         name: String::from("Ferris"),
22         arms: 2,
23         legs: 4,
24         weapon: String::from("claw"),
25     };
26
27     match ferris.species {
28         Species::Crab => println!("{}", is a crab",ferris.name),
29         Species::Octopus => println!("{}", is a octopus",ferris.name),
30         Species::Fish => println!("{}", is a fish",ferris.name),
31         Species::Clam => println!("{}", is a clam",ferris.name),
32     }
33 }

```

2.3.3.1 带数据的枚举

`enum` 的元素可以有一个或多个数据类型，从而使其表现得像 C 语言中的联合。

当使用 `match` 对一个 `enum` 进行模式匹配时，可以将变量名称绑定到每个数据值。

`enum` 的内存细节：

- 枚举数据的内存大小等于它最大元素的大小。此举是为了让所有可能的值都能存入相同的内存空间。
- 除了元素数据类型（如果有）之外，每个元素还有一个数字值，用于表示它是哪个标签。

其他细节：

- Rust 的 `enum` 也被称为 **标签联合**（tagged-union）
- 把类型组合成一种新的类型，这就是人们所说的 Rust 具有 **代数类型** 的含义。

```

1  #![allow(dead_code)] // prevents compiler warnings
2
3  enum Species { Crab, Octopus, Fish, Clam }
4  enum PoisonType { Acidic, Painful, Lethal }
5  enum Size { Big, Small }
6  enum Weapon {
7      Claw(i32, Size),
8      Poison(PoisonType),
9      None
10 }
11
12 struct SeaCreature {
13     species: Species,
14     name: String,
15     arms: i32,
16     legs: i32,
17     weapon: Weapon,
18 }
19
20 fn main() {
21     // SeaCreature's data is on stack
22     let ferris = SeaCreature {
23         // String struct is also on stack,
24         // but holds a reference to data on heap
25         species: Species::Crab,
26         name: String::from("Ferris"),
27         arms: 2,
28         legs: 4,

```

```

29         weapon: Weapon::Claw(2, Size::Small),
30     };
31
32     match ferris.species {
33         Species::Crab => {
34             match ferris.weapon {
35                 Weapon::Claw(num_claws,size) => {
36                     let size_description = match size {
37                         Size::Big => "big",
38                         Size::Small => "small"
39                     };
40                     println!("ferris is a crab with {}-{} claws", num_claws,
size_description)
41                 },
42                 _ => println!("ferris is a crab with some other weapon")
43             }
44         },
45         _ => println!("ferris is some other animal"),
46     }
47 }

```

2.3.4 Vectors

和C++中类似，`vector` 是可变长度的元素集合，以 `Vec` 结构表示，可以使用 `vec!` 宏快速创建 `vector`。

`Vec` 有一个形如 `iter()` 的方法可以为一个 `vector` 创建迭代器，这可以轻松地将 `vector` 用到 `for` 循环中去。

一些细节：

- `Vec` 是一个结构体，但是内部其实保存了在堆上固定长度数据的引用。
- 一个 `vector` 开始有默认大小容量，当更多的元素被添加进来后，它会重新在堆上分配一个新的并具有更大容量的定长列表。（类似 C++ 的 `vector`）

```

1  fn main() {
2      let mut i32_vec = Vec::<i32>::new();
3      i32_vec.push(1);
4      i32_vec.push(2);
5      i32_vec.push(3);
6
7      let mut float_vec = Vec::new();
8      float_vec.push(1.3);
9      float_vec.push(2.3);
10     float_vec.push(3.4);
11 }

```

```

12     let string_vec = vec![String::from("Hello"), String::from("World")];
13
14     for word in string_vec.iter() {
15         println!("{}", word);
16     }
17 }

```

2.4 泛型

泛型是什么？

泛型允许我们不完全定义一个 `struct` 或 `enum`，使编译器能够根据我们的代码使用情况，在编译时创建一个完全定义的版本。

Rust 通常可以通过查看实例来推断出最终的类型，但是可以使用 `::<T>` 操作符来显式地进行操作，该操作符也被称为 `turbofish`。

```

1 struct BagOfHolding<T> {
2     item: T,
3 }
4
5 fn main() {
6     // 使用泛型，会在编译时创建的实际的类型，导致代码膨胀
7     let i32_bag = BagOfHolding::<i32> { item: 42 };
8     let bool_bag = BagOfHolding::<bool> { item: true };
9
10    // Rust可以推断出泛型的类型
11    let float_bag = BagOfHolding { item: 3.14 };
12
13    let bag_in_bag = BagOfHolding {
14        item: BagOfHolding { item: "嘭! " },
15    };
16
17    println!("{}",
18        i32_bag.item, bool_bag.item, float_bag.item, bag_in_bag.item.item
19    );
20 }
21

```

2.4.1.1 泛型枚举

内置的泛型枚举 `Option`

可以让我们不使用 `null` 就可以表示可以为空的值。


```

1 enum Option<T> {
2     None,
3     Some(T),
4 }

```

这个枚举很常见，使用关键字 `Some` 和 `None` 可以在任何地方创建其实例。

```

1
2 struct BagOfHolding<T> {
3     item: Option<T>,
4 }
5
6 fn main() {
7
8     let i32_bag = BagOfHolding::<i32> { item: None };
9
10    if i32_bag.item.is_none() {
11        println!("there's nothing in the bag!");
12    } else {
13        println!("there's something in the bag!");
14    }
15
16    let i32_bag = BagOfHolding::<i32> { item: Some(42) };
17
18    if i32_bag.item.is_some() {
19        println!("there's something in the bag!");
20    } else {
21        println!("there's nothing in the bag!");
22    }
23
24    // match 可以优雅地解构 Option，并确保处理了所有的可能情况
25    match i32_bag.item {
26        Some(v) => println!("found {} in bag!", v),
27        None => println!("found nothing"),
28    }
29 }

```

内置的泛型枚举 `Result`

可以返回一个可能包含错误的值。

```

1 enum Result<T, E> {
2     Ok(T),
3     Err(E),

```

```

4 }
5
6 fn do_something_that_might_fail(i:i32) -> Result<f32,String> {
7     if i == 42 {
8         Ok(13.0)
9     } else {
10        Err(String::from("this is not the right number"))
11    }
12 }
13
14 fn main() {
15     let result = do_something_that_might_fail(12);
16
17     // match 让我优雅地解构 Rust, 并且确保我们处理了所有情况!
18     match result {
19         Ok(v) => println!("found {}", v),
20         Err(e) => println!("Error: {}",e),
21     }
22 }

```

这个枚举很常见, 使用关键字 `Ok` 和 `Err` 可以在任何地方创建其实例。

`main` 函数有可以返回 `Result` 的能力

```

1 fn main() -> Result<(), String> {
2     let result = do_something_that_might_fail(12);
3
4     match result {
5         Ok(v) => println!("found {}", v),
6         Err(_e) => {
7             // 优雅地处理错误
8
9             // 返回一个说明发生了什么的错误!
10            return Err(String::from("something went wrong in main!"));
11        }
12    }
13
14    // Notice we use a unit value inside a Result Ok
15    // to represent everything is fine
16    Ok(())
17 }

```

`Result` 还有个强大的操作符 `?` 来与之配合。

```

1 fn do_something_that_might_fail(i: i32) -> Result<f32, String> {
2     if i == 42 {
3         Ok(13.0)
4     } else {
5         Err(String::from("this is not the right number"))
6     }
7 }
8
9 fn main() -> Result<(), String> {
10    // 看看我们节省了多少代码!
11    let v = do_something_that_might_fail(42)?;
12
13    //match do_something_that_might_fail() {
14    //    Ok(v) => v,
15    //    Err(e) => return Err(e),
16    //}
17
18    println!("found {}", v);
19    Ok(())
20 }
21

```

当只想快速的实现代码逻辑的时候，不想过多的处理错误逻辑，可以通过 `unwrap()` 来处理 `unwrap` 的功能：

1. 获取 Option/Result 内部的值
2. 如果枚举的类型是 None/Err，则会 `panic!`

这两段代码是等价的：

```

1 my_option.unwrap()

```

```

1 match my_option {
2     Some(v) => v,
3     None => panic!("some error message generated by Rust!"),
4 }

```

类似的：

```

1 my_result.unwrap()

```

```

1  match my_result {
2      Ok(v) => v,
3      Err(e) => panic!("some error message generated by Rust!"),
4  }

```

2.5 Owership 所有权

相较于其他编程语言，Rust 具有一套独特的内存管理范例，就是**所有权和生命周期**。

2.5.1 所有权

实例化一个类型并且将其**绑定**到变量名上将会创建一些内存资源，而这些内存资源将会在其整个**生命周期**中被 Rust 编译器检验。被绑定的变量即为该资源的**所有者**。

```

1  struct Foo {
2      x: i32,
3  }
4
5  fn main() {
6      // 实例化这个结构体并将其绑定到具体的变量上来创建内存资源
7      let foo = Foo { x: 42 };
8      // foo 即为该资源的所有者
9
10     let foo_a = Foo { x: 42 };
11     let foo_b = Foo { x: 13 };
12
13     println!("{}", foo_a.x);
14     // foo_a 将在这里被 dropped 因为其在这之后再也没有被使用
15
16     println!("{}", foo_b.x);
17     // foo_b 将在这里被 dropped 因为这是函数域的结尾
18 }

```

Rust 将使用资源最后被使用的位置或者一个函数域的结束来作为资源被析构和释放的地方。此处析构和释放的概念被称之为 **drop**（丢弃）。

细节：

- Rust 没有垃圾回收机制。
- 在 C++ 中，这被也称为“资源获取即初始化”（RAII）。

分级释放

```

1 struct Bar {
2     x: i32,
3 }
4
5 struct Foo {
6     bar: Bar,
7 }
8
9 fn main() {
10     let foo = Foo { bar: Bar { x: 42 } };
11     println!("{}", foo.bar.x);
12     // foo 首先被 dropped 释放
13     // 紧接着是 foo.bar
14 }

```

释放是分级进行的, 删除一个结构体时, 结构体本身会先被释放, 紧接着才分别释放相应的子结构体并以此类推。

内存细节:

- Rust 通过自动释放内存来帮助确保减少内存泄漏。
- 每个内存资源仅会被释放一次。

移交所有权

将所有者作为参数传递给函数时, 其所有权将移交至该函数的参数。在一次**移动**后, 原函数中的变量将无法再被使用。

```

1 struct Foo {
2     x: i32,
3 }
4
5 fn do_something(f: Foo) {
6     println!("{}", f.x);
7     // f 在这里被 dropped 释放
8 }
9
10 fn main() {
11     let foo = Foo { x: 42 };
12     // foo 被移交至 do_something
13     do_something(foo);
14     // 此后 foo 便无法再被使用
15 }
16

```

内存细节:

- 在**移动**期间,所有者的堆栈值将会被复制到函数调用的参数堆栈中。

归还

所有权也可以从一个函数中被归还

```
1 struct Foo {
2     x: i32,
3 }
4
5 fn do_something() -> Foo {
6     Foo { x: 42 }
7     // 所有权被移出
8 }
9
10 fn main() {
11     let foo = do_something();
12     // foo 成为了所有者
13     // foo 在函数域结尾被 dropped 释放
14 }
```

引用

引用允许我们通过 `&` 操作符来借用对一个资源的访问权限。引用也会如同其他资源一样被释放

```
1 struct Foo {
2     x: i32,
3 }
4
5 fn main() {
6     let foo = Foo { x: 42 };
7     let f = &foo;
8     println!("{}", f.x);
9     // f 在这里被 dropped 释放
10    // foo 在这里被 dropped 释放
11 }
```

可变的引用

使用 `&mut` 操作符来借用对一个资源的可变访问权限。在发生了可变借用后,一个资源的所有者便不可以再次被借用或者修改。

内存细节：

- Rust 之所以要避免同时存在两种可以改变所拥有变量值的方式，是因为此举可能会导致潜在的数据争用（data race）。

```
1 struct Foo {
2     x: i32,
3 }
4
5 fn do_something(f: Foo) {
6     println!("{}", f.x);
7     // f 在这里被 dropped 释放
8 }
9
10 fn main() {
11     let mut foo = Foo { x: 42 };
12     let f = &mut foo;
13
14     // 会报错: do_something(foo);
15     // 因为 foo 已经被可变借用而无法取得其所有权
16
17     // 会报错: foo.x = 13;
18     // 因为 foo 已经被可变借用而无法被修改
19
20     f.x = 13;
21     // f 会因此后不再被使用而被 dropped 释放
22
23     println!("{}", foo.x);
24
25     // 现在修改可以正常进行因为其所有可变引用已经被 dropped 释放
26     foo.x = 7;
27
28     // 移动 foo 的所有权到一个函数中
29     do_something(foo);
30 }
```

解引用

使用 `&mut` 引用时, 通过 `*` 操作符来修改其指向的值。

也可以使用 `*` 操作符来对所拥有的值进行拷贝（前提是该值可以被拷贝——我们将会在后续章节中讨论可拷贝类型）

```
1 let mut foo = 42;
2 let f = &mut foo;
```

```

3 let bar = *f; // 取得所有者值的拷贝
4 *f = 13;      // 设置引用所有者的值
5 println!("{}", bar);
6 println!("{}", foo);
7 }

```

这里就和c/cpp 中的概念很像了

```

1 struct Foo {
2     x: i32,
3 }
4
5 fn do_something(f: &mut Foo) {
6     f.x += 1;
7     // 可变引用 f 在这里被 dropped 释放
8 }
9
10 fn main() {
11     let mut foo = Foo { x: 42 };
12     do_something(&mut foo);
13     // 因为所有的可变引用都在 do_something 函数内部被释放了
14     // 此时我们便可以再创建一个
15     do_something(&mut foo);
16     // foo 在这里被 dropped 释放
17 }

```

Rust 对于引用的规则的简单总结：

- Rust 只允许同时存在一个可变引用**或者**多个不可变引用，**不许可**可变引用和不可变引用同时存在。
- 一个引用永远也不会比它的所有者存活得更久。

而在函数间进行引用的传递时，以上这些通常都不会成为问题。

内存细节：

- 上面的第一条规则避免了数据争用的出现。
- 而第二条引用规则则避免了通过引用而错误的访问到不存在的数据(C 语言中被称为悬垂指针)

2.5.2 5.2 生命周期

2.5.2.1 显式生命周期

尽管 Rust 不总是在代码中将它展示出来，但编译器会理解每一个变量的生命周期并进行验证以确保一个引用不会有长于其所有者的存在时间。

同时，函数可以通过使用一些符号来参数化函数签名，以帮助界定哪些参数和返回值共享同一生命周期。

生命周期注解总是以 `'` 开头，例如 `'a`，`'b` 以及 `'c`。

```
1 struct Foo {
2     x: i32,
3 }
4
5 // 参数 foo 和返回值共享同一生命周期
6 fn do_something<'a>(foo: &'a Foo) -> &'a i32 {
7     return &foo.x;
8 }
9
10 fn main() {
11     let mut foo = Foo { x: 42 };
12     let x = &mut foo.x;
13     *x = 13;
14     // x 在这里被 dropped 释放从而允许我们再创建一个不可变引用
15     let y = do_something(&foo);
16     println!("{}", y);
17     // y 在这里被 dropped 释放
18     // foo 在这里被 dropped 释放
19 }
```

生命周期注解可以通过区分函数签名中不同部分的生命周期，来允许我们显式地明确某些编译器靠自己无法解决的场景。

```
1 struct Foo {
2     x: i32,
3 }
4
5 // foo_b 和返回值共享同一生命周期
6 // foo_a 则拥有另一个不相关联的生命周期
7 fn do_something<'a, 'b>(foo_a: &'a Foo, foo_b: &'b Foo) -> &'b i32 {
8     println!("{}", foo_a.x);
9     println!("{}", foo_b.x);
10    return &foo_b.x;
11 }
12
13 fn main() {
14     let foo_a = Foo { x: 42 };
15     let foo_b = Foo { x: 12 };
16     let x = do_something(&foo_a, &foo_b);
```

```

17 // foo_a 在这里被 dropped 释放因为只有 foo_b 的生命周期在此之后还在延续
18 println!("{}", x);
19 // x 在这里被 dropped 释放
20 // foo_b 在这里被 dropped 释放
21 }

```

2.5.2.2 静态生命周期

一个静态变量是一个在编译期间即被创建并存在于整个程序始末的内存资源。他们必须被明确指定类型。

一个**静态生命周期**是指一段内存资源无限期地延续到程序结束。需要注意的一点是，在此定义之下，一些静态生命周期的资源也可以在运行时被创建。

拥有静态生命周期的资源会拥有一个特殊的生命周期注解 `'static`。

`'static` 资源永远也不会被 **drop** 释放。如果静态生命周期资源包含了引用，那么这些引用的生命周期也一定是 `'static` 的。（任何缺少了此注解的引用都不会达到同样长的存活时间）

内存细节：

- 因为静态变量可以全局性地被任何人访问读取而潜在地引入数据争用，所以修改它具有内在的危险性。我们会在稍后讨论使用全局数据的一些挑战。
- Rust 允许使用 `unsafe { ... }` 代码块来进行一些无法被编译器担保的内存操作

```

1 static PI: f64 = 3.1415;
2
3 fn main() {
4     // 静态变量的范围也可以被限制在一个函数内
5     static mut SECRET: &'static str = "swordfish";
6
7     // 字符串面值拥有 'static 生命周期
8     let msg: &'static str = "Hello World!";
9     let p: &'static f64 = &PI;
10    println!("{}", msg, p);
11
12    // 你可以打破一些规则，但是必须是显式地
13    unsafe {
14        // 我们可以修改 SECRET 到一个字符串面值因为其同样是 'static 的
15        SECRET = "abracadabra";
16        println!("{}", SECRET);
17    }
18 }

```

数据类型中的生命周期

和函数相同，数据类型也可以用生命周期注解来参数化其成员。 Rust 会验证引用所包含的数据结构永远也不会比引用指向的所有者存活周期更长。 我们不能在运行中拥有一个包括指向虚无的引用结构存在！